

AI CHATBOX APPLICATION

Real-Time Conversational AI

Built with React · Redux Toolkit · Node.js · AI API

Presented by Priyanshi Dabral

The logo for Navgurukul, featuring the word "nav" in red and "gurukul" in black, with a stylized orange and red symbol above the "g".

Project Overview

What I Built

- A real-time chatbox application
- Integrated with an AI API
- Built using React + Redux Toolkit
- Backend proxy with Node.js / Express

Project Goal

- Conversational interface where users send messages and receive AI responses
- Manage state efficiently with Redux Toolkit
- Handle async API calls cleanly
- Smooth, responsive user experience

3

Async States Managed

AI

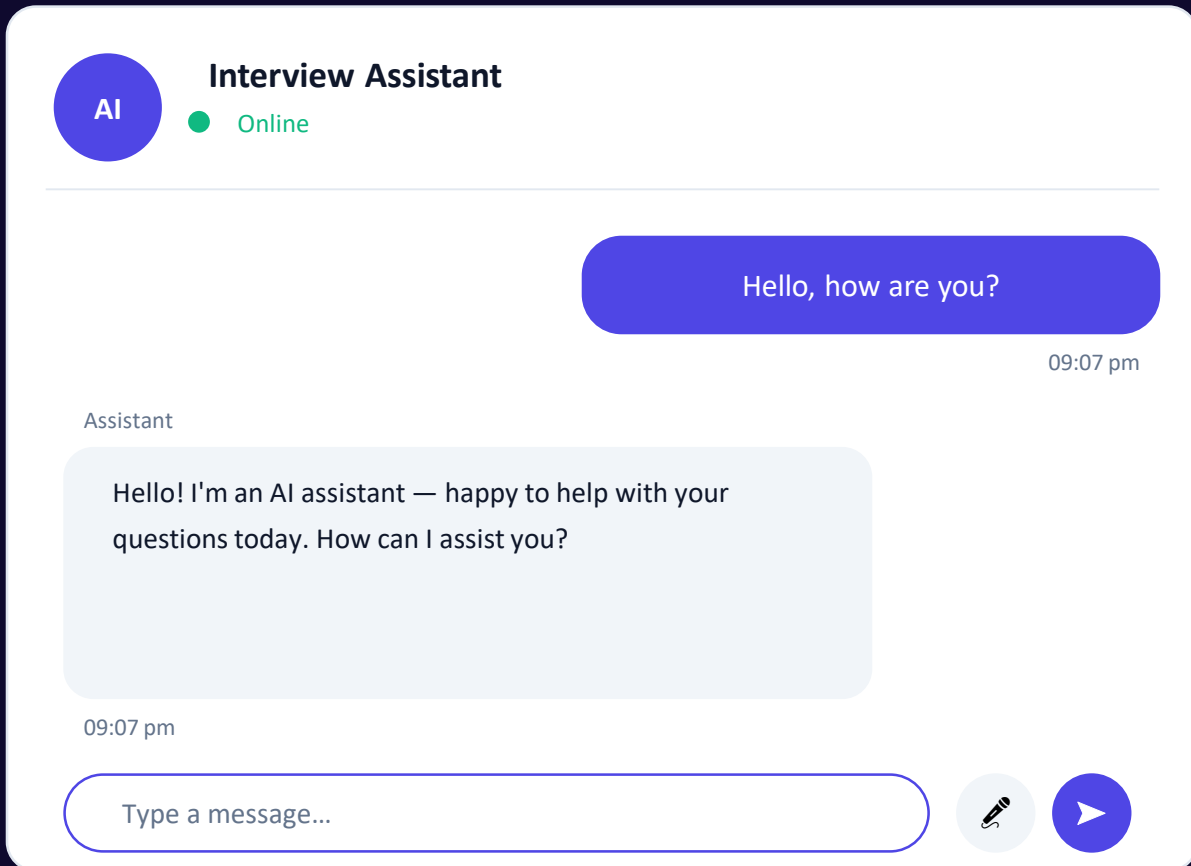
Real-Time Responses

100%

Secure API Key

Interface & Experience

Clean, minimal, and responsive across devices.



Clean & Minimal

Distraction-free conversational layout.



Clear Message Separation

User and AI bubbles are visually distinct.



Send + Mic Input

Quick text or voice options for messages.



Typing Indicator

Loading state while waiting for AI reply.



Fully Responsive

Works smoothly across screen sizes.

Technologies Used



React

Component-based UI



Redux Toolkit

Global state management



createAsyncThunk

Async API lifecycle



Node.js / Express

Backend proxy server



AI API

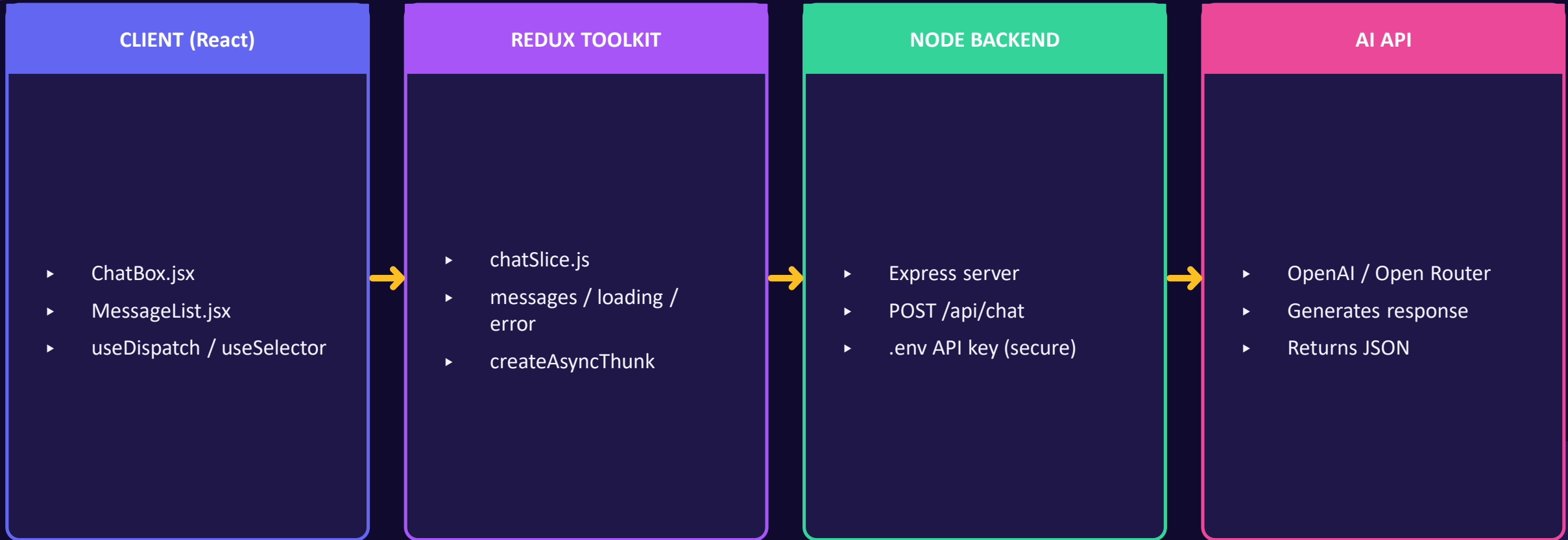
Conversational AI engine



CSS Modules

Scoped styling

End-to-End Data Flow



User types → React dispatches thunk → Redux pending → Backend proxy → AI API → Response → Redux fulfilled → UI updates

Redux Toolkit Slice

State Shape

- messages[] — chat history
- loading — show typing indicator
- error — handle API failure

Lifecycle

- ▶ pending → set loading = true
- ▶ fulfilled → push AI message
- ▶ rejected → set error message

```
// chatSlice.js
const chatSlice = createSlice({
  name: 'chat',
  initialState: {
    messages: [],
    loading: false,
    error: null,
  },
  reducers: {
    addUserMessage: (state, action) => {
      state.messages.push(action.payload);
    },
  },
  extraReducers: (b) => {
    b.addCase(sendMessage.pending, s => { s.loading = true; })
    .addCase(sendMessage.fulfilled, (s, a) => {
      s.loading = false;
      s.messages.push(a.payload);
    })
    .addCase(sendMessage.rejected, (s, a) => {
      s.loading = false;
      s.error = a.error.message;
    });
  },
});
```

createAsyncThunk

Why createAsyncThunk?

- Auto-generates pending / fulfilled / rejected actions
- Keeps async logic out of components
- Centralized error handling
- Cleaner than useEffect + useState



```
// chatThunks.js
export const sendMessage = createAsyncThunk(
  'chat/sendMessage',
  async (userMessage, thunkAPI) => {
    try {
      const res = await fetch('/api/chat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: userMessage }),
      });
      if (!res.ok) throw new Error('API failed');
      const data = await res.json();
      return { sender: 'AI', text: data.reply };
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message);
    }
  }
);
```

Node.js / Express Server

Why a Backend?

- Hides API key from the browser
- Avoids CORS issues with AI provider
- Central place for rate-limiting, logging
- Easier to swap AI providers later



```
// server/index.js
import express from 'express';
const app = express();
app.use(express.json());

app.post('/api/chat', async (req, res) => {
  const { message } = req.body;
  const r = await fetch(AI_URL, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.AI_KEY}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ prompt: message }),
  });
  const data = await r.json();
  res.json({ reply: data.output });
});

app.listen(5000);
```


ChatBox.jsx

Responsibilities

- Reads messages from Redux store
- Dispatches sendMessage thunk on submit
- Shows loading indicator
- Renders error state



```
// ChatBox.jsx
function ChatBox() {
  const dispatch = useDispatch();
  const { messages, loading } =
    useSelector(s => s.chat);
  const [input, setInput] = useState('');

  const onSend = () => {
    dispatch(addUserMessage({sender:'You',text:input}));
    dispatch(sendMessage(input));
    setInput('');
  };

  return (
    <div className="chat">
      <MessageList messages={messages} />
      {loading && <Typing />}
      <input value={input}
        onChange={e=>setInput(e.target.value)} />
      <button onClick={onSend}>Send</button>
    </div>
  );
}
```

Problems Solved

Async State

Managing pending/fulfilled/rejected without messy useEffect

API Security

Keeping the AI key off the client via backend proxy

Error Handling

Graceful UI for failed API calls and timeouts

UX Smoothness

Loading indicators + auto-scroll on new messages

Project Structure

Clean separation: Frontend (src) • Backend (Express proxy) • Redux state • API services



```
chatbot/
├── backend/
│   ├── .env                ← AI_KEY
│   └── server.js           ← Express proxy
├── src/
│   ├── components/
│   │   ├── ChatBox.jsx
│   │   ├── ChatBox.module.css
│   │   ├── InputBox.jsx
│   │   └── Message.jsx
│   ├── redux/
│   │   ├── chatSlice.js
│   │   └── store.js
│   ├── services/
│   │   └── api.js          ← Axios calls
│   ├── App.jsx
│   └── index.js
├── package.json
└── README.md
```

Separation of Concerns

- components/ → pure UI
- redux/ → state + async logic
- server/ → backend proxy + secrets
- Clean, scalable folder structure
- Easy to onboard new developers

Step-by-Step Flow

1

User types a message and clicks Send

2

Component dispatches sendMessage thunk

3

Redux sets loading = true (typing shown)

4

Backend proxy forwards request to AI API

5

AI returns response → fulfilled action fires

6

Reducer pushes AI message → UI re-renders

Future Enhancements & Scalability

Where this project can go next — features, scale, and production hardening.



Multi-User Scalability

Handle 1000s of concurrent users



Feature Enhancements

Streaming, history, themes



Production Hardening

Auth, rate-limit, monitoring



Quality & Testing

Unit, integration, E2E tests

Handling Multiple Concurrent Users

Currently the app handles a single user. For lakhs (100K+) of concurrent users:

Load Balancing

Nginx / AWS ALB to distribute traffic across multiple Node instances

Horizontal Scaling

Run backend on Kubernetes / Docker Swarm — auto-scale on CPU/RAM

Stateless Backend

No in-memory session storage; use Redis for shared session state

Database Layer

MongoDB / PostgreSQL for chat history, with read replicas + sharding

Caching

Redis for hot AI responses + CDN (CloudFront) for static React build

Queue System

Kafka / RabbitMQ to buffer AI requests and prevent provider rate-limits

Planned Feature Enhancements

Chat History Persistence

Save messages per user in DB — resume conversations across sessions

Streaming AI Responses

Server-Sent Events (SSE) — words appear as AI generates, not after full reply

Multi-Conversation Support

Sidebar with conversation list, like ChatGPT — switch between threads

Voice Input / Output

Web Speech API for hands-free chat

File / Image Upload

Multimodal — let users attach images for AI to analyze

Themes & Personalization

Dark/light mode, custom AI persona, user preferences

Security & Reliability Roadmap

User Authentication

JWT / OAuth login — track per-user usage and history

Rate Limiting

express-rate-limit — prevent abuse and control AI API costs

Input Sanitization

Validate + escape user input to prevent prompt injection

Monitoring & Logs

Sentry for errors, Datadog/Grafana for metrics & uptime

CI/CD Pipeline

GitHub Actions → automated tests → deploy to AWS/Vercel

Cost Optimization

Cache repeated prompts; smaller models for simple queries

Testing Strategy (Future)

Unit Tests — Jest

Test reducers, thunks, utility functions in isolation

Component Tests — RTL

React Testing Library for ChatBox & MessageList rendering

Integration Tests

Mock the backend; verify dispatch → state → UI cycle

E2E Tests — Cypress / Playwright

Full user journey: type → send → receive AI reply

What I Took Away

- ✓ Real-world Redux Toolkit state management
- ✓ Async API integration with createAsyncThunk
- ✓ Backend proxy pattern for API key security
- ✓ Building scalable frontend architectures
- ✓ Designing for future scale & multi-user systems

Thank You

Questions & Discussion

— Priyanshi Dabral —